Map-Reflect: Uncertainty-Guided Reflection for Schema Mapping

Balaji Ganesan*
IBM Research
India
bganesa1@in.ibm.com

Avirup Saha*
IBM Research
India
avirup.saha2@ibm.com

Gunjan Siddharth
Vellore Institute of Technology
India
gunjan.siddharth2022@vitstudent.ac.in

Ravindran Jagannathan
IBM Software
USA
ravi.jagannathan@ibm.com

Amit Paradkar
IBM Research
USA
paradkar@us.ibm.com

Pratibha Moogi
IBM Research
India
pratibha.moogi@ibm.com

ABSTRACT

We study many-to-many schema mapping, a setting where multiple source types/fields may correspond to overlapping targets and vice versa—cases that retrievers and single-chain API sequencers fail to disambiguate. Prior approaches use self-reflection where models critique their own output, but self-reflection lacks external grounding and models may reinforce their own errors. We introduce Map-Reflect, an *induced reflection* framework that performs schema mapping through a two-phase approach: an initial greedy mapping followed by selective reflection triggered by external signals. Our approach is schema-agnostic, applying uniformly to REST APIs, databases, and other structured schemas. Map-Reflect outperforms all baselines across API schema benchmarks (ITOps-Graph, NL2GQL, ToolBench) and achieves the highest type-level accuracy on relational database schemas (OMOP, SchemaNet). On SchemaNet field-level accuracy, AnyTool leads, though it requires building a static three-tier API hierarchy before deployment. A central finding from our ablation is that *not all reflection signals are equally reliable*: uncertainty quantification (UQ) consistently improves mapping quality by selectively targeting genuinely ambiguous predictions, while error-based and neighbourhood-based signals fire too broadly and degrade accuracy by 7–15 pp. We therefore recommend UQ-triggered reflection as the principled default. This recommendation is theoretically grounded in the consistency hypothesis [41] and the SIMBA framework [2], which establish that generation consistency is a reliable proxy for correctness in structured-output tasks analogous to schema field prediction. The framework emits interpretable rationales through a wiki validated by business stakeholders for human-in-the-loop validation in enterprise settings.

*Equal contribution.

PVLDB Reference Format:
Balaji Ganesan*, Avirup Saha*, Gunjan Siddharth, Ravindran Jagannathan, Amit Paradkar, and Pratibha Moogi. Map-Reflect: Uncertainty-Guided Reflection for Schema Mapping. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.

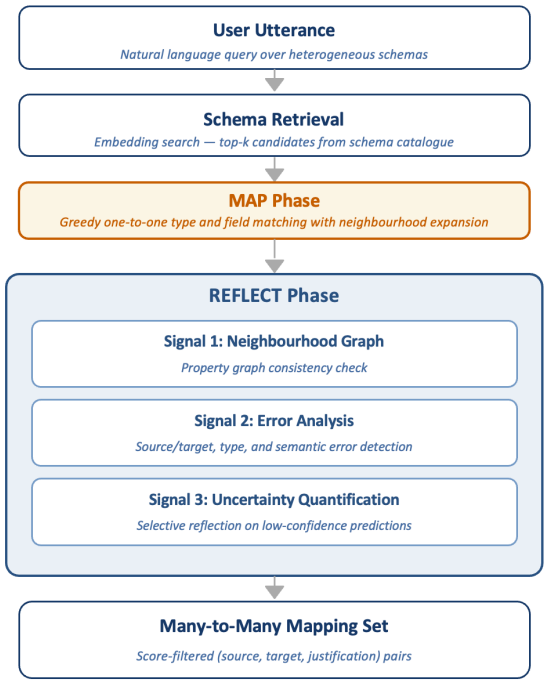


Figure 1: Map-Reflect pipeline: greedy MAP followed by signal-driven REFLECT.

1 INTRODUCTION

Organizations rely on diverse API ecosystems and databases that must be integrated for effective data access. Schema mapping—identifying equivalent entity types across different schemas—is critical for building unified data access layers where users query heterogeneous sources without knowledge of underlying schema differences. This capability is increasingly important for AI agents and tool-calling scenarios, where LLMs must select and invoke APIs across multiple services.

Traditional approaches focus on database integration with column-level mapping, but enterprise environments require schema retrieval and multi-faceted mapping across both APIs and

doi:XX.XX/XXX.XX

databases. For example, Site Reliability Engineers managing hybrid cloud environments must query across multiple observability platforms, each with distinct schemas. The complexity of mapping across these disparate systems creates significant overhead.

The Many-to-Many Challenge. Prior work on schema mapping focuses on one-to-one alignments or single-chain API sequencing. Real enterprise environments require *many-to-many mappings* where multiple source types/fields correspond to overlapping target structures. For example, `Incident` from one system may correctly map to both `Alert` and `Event` in another. Greedy one-to-one mapping leaves too many elements unmapped, while naive approaches create spurious mappings that break query semantics. A key challenge for any reflection mechanism is knowing *when* to reflect: firing unconditionally corrupts correct predictions, while a reliable trigger must identify genuinely uncertain cases. Uncertainty quantification provides this gate, and our central empirical finding is that UQ-triggered reflection is the only signal that consistently improves accuracy across datasets.

Recent advances in tool calling with large language models show promise for API-related tasks. ToolLLM [32] introduced the ToolBench dataset with a BERT-based neural API retriever while AnyTool [8] leverages GPT-4 [29]'s function calling with hierarchical API organization and self-reflection. Sequential function calling with variable mapping across APIs has been studied by Saha et al. [34], who introduce GraphQLRestBench and demonstrate that GraphQL schema representations are effective for encoding API response structure in LLM prompts for multi-step API sequencing. However, these methods focus on sequencing a single chain of calls; they do not address systematic alignment of schema elements across products, which is the prerequisite for variable mapping between function calls. Critically, self-reflection mechanisms lack external grounding—the model critiques its own output without validation against external signals, potentially reinforcing errors. Further, these approaches require expensive models like GPT-4, limiting practical applicability.

The Preprocessing Cost Problem. The deployment contexts for schema mapping make preprocessing costs a genuine practical concern, not a one-time fixed overhead.

(1) Data Fabric and Data Mesh architectures. Enterprise Data Fabric and Data Mesh implementations [3] build a unified Data Access Layer by federating schemas across disparate systems. These environments grow continuously—through M&A activity, cloud migration, and onboarding of new SaaS services—so the schema inventory is never static. Each newly acquired or onboarded system requires its APIs and databases to be mapped into the fabric. AnyTool [8], for example, organises APIs into a three-tier hierarchy (meta-agent → category-agents → tool-agents) built around RapidAPI's static category taxonomy. Adding a new enterprise product requires extending this taxonomy, writing specialised bootstrap prompts for each new agent tier, and re-running the agent hierarchy—it is emphatically not a one-time cost. Similarly, ColBERT-v2 multi-vector indexes must be rebuilt and MatchMaker's DSPy bootstrapping loop must be re-run whenever the schema set changes.

(2) Backend for Frontend and function calling. A primary use case for schema mapping is enabling downstream function calling and API sequencing in developer tooling. Once source-to-target schema correspondences are established, an agent can sequence API calls across products to satisfy a user intent [34]. This connection is direct: Map-Reflect's output—a field-level mapping between two schemas—is precisely the variable mapping that sequential function calling requires [34]. Backend for Frontend (BFF) patterns expose unified APIs to client applications, requiring schema alignment every time a new upstream service is added. In developer workflows, new APIs are onboarded frequently; preprocessing pipelines that must be re-executed per schema addition create friction that slows adoption.

(3) Agentic code tooling. Code agents such as IBM Bob, Claude Code, and Gemini CLI are increasingly expected to autonomously discover, invoke, and compose APIs from a live catalogue. These agents need schema mapping as a *runtime capability*—discovering and aligning schemas on demand as new tools are registered—rather than as a batch offline step. An approach that requires building a multi-vector index, running a DSPy bootstrapping loop, or constructing a three-tier agent hierarchy before any mapping can be performed is incompatible with runtime tool registration. Even at inference time, AnyTool [8] issues an average of 43.3 GPT-4 API calls per query [8], making it prohibitively expensive for the high-frequency schema discovery that agentic tooling requires.

Map-Reflect takes the opposite position: retrieval is intentionally lightweight (hybrid dense and lexical search), and imperfect retrieval is tolerated because the Reflect phase corrects rank errors through externally grounded signals. New schemas require only parsing and optional RAG enrichment—no index rebuilds, no bootstrapping loops, no graph construction or model retraining. This design directly supports dynamic schema registration in Data Fabric, BFF, and agentic code tooling contexts.

Building a unified Data Access Layer requires mapping schemas across both APIs and databases. We address this challenge through experiments on API datasets (ITOpsGraph, NL2GQL, ToolBench) and database schemas (OMOP, SchemaNet). Our ITOpsGraph benchmark provides schema mapping across heterogeneous enterprise APIs, featuring (i) deeply nested type hierarchies, (ii) domain-specific terminology where semantically equivalent concepts have different names, and (iii) many-to-many mappings where a single source type correctly maps to multiple targets. Our wiki-based SME annotation provides continuous ground truth refinement.

We introduce Map-Reflect, an *induced reflection* framework that addresses these limitations through a two-phase approach (Figure 1). In the first *Map* phase, we greedily find the best mapping for each source type and its fields. In the second *Reflect* phase, we explore three candidate external signals to induce reflection: (1) a *neighbourhood graph* that validates mapping assumptions, (2) *error analysis* from LLM-as-judge evaluation, schema deployment, and query execution, and (3) *uncertainty quantification* that selectively targets genuinely ambiguous predictions. Our ablation shows that UQ-triggered reflection is the only signal that consistently improves accuracy across both API and database benchmarks, while error-based and neighbourhood-based signals

degrade performance by 7–15 pp. We therefore recommend UQ-triggered reflection as the principled default and provide theoretical grounding in the consistency hypothesis [41] and SIMBA [2].

Our contributions are: (1) Map-Reflect, a two-phase induced reflection framework for many-to-many schema mapping that explores three external signals (neighbourhood graph, error analysis, uncertainty quantification) and establishes through ablation that *UQ-triggered reflection is the only signal that reliably improves accuracy*—error-based and neighbourhood-based signals fire too broadly and hurt by 7–15 pp, (2) a theoretical grounding of UQ-triggered reflection in the consistency hypothesis [41] and the SIMBA aggregation framework [2], which predict that generation consistency is a reliable correctness proxy for structured-output tasks such as schema field prediction, (3) a lightweight hybrid retrieval design (dense + BM25, merged by Reciprocal Rank Fusion) that requires no offline schema preprocessing, with reflection absorbing residual retrieval errors, (4) a wiki-based human-in-the-loop system for continuous ground truth refinement with full provenance tracking, (5) a new enterprise API schema mapping benchmark (ITOpsGraph) and comprehensive evaluation on both API datasets (ITOpsGraph, NL2GQL, ToolBench) and relational database datasets (OMOP, SchemaNet), demonstrating that Map-Reflect achieves the best type-level accuracy across all benchmarks and the best field-level accuracy on API schemas, while AnyTool leads on SchemaNet field-level accuracy at the cost of requiring offline hierarchy construction.

2 RELATED WORK

ToolLLM [32] introduced the ToolBench dataset with an API Retriever for natural language instructions. AnyTool [8] incorporates self-reflection for reliability via a three-tier agent hierarchy (meta-agent → category-agents → tool-agents) built around RapidAPI's static category taxonomy, and is hardwired to GPT-4 function calling. The hierarchy cannot be transferred to a new API corpus without rebuilding the taxonomy from scratch. AnyTool's self-reflection also lacks external grounding—the model critiques its own output without validation against type-system checks or uncertainty measures, potentially reinforcing errors. These approaches focus on API sequencing, not schema mapping.

Classical Schema Matching. Cupid [24], COMA [7], and iMAP [6] established multi-signal matching combining linguistic, structural, and domain-specific similarity. However, these systems assume flat relational schemas and produce one-to-one correspondences, limiting applicability to nested API schemas with many-to-many relationships. Map-Reflect builds on multi-signal intuition but uses LLM-driven reasoning with externally grounded reflection.

DynaMap [27], embedding-based approaches [12], and SMUTF [43] use dynamic programming, word vectors, and XGBoost classifiers respectively, but struggle with contextual relationships.

API retrieval and sequencing have been addressed by embedding-based retrievers [18, 31], fine-tuned methods [32], graph-based approaches [26], and GraphQL-based sequential function calling [34]. Pre-LLM API mapping work includes document-based [22], knowledge graph [20], and GNN approaches [36].

LLM-Based Schema Mapping. LSM [42] uses fine-tuned BERT encoders; Magneto [21] combines SLMs with LLM reranking; ReMatch [37] and GraphRAGToolFusion [23] add retrieval mechanisms. MatchMaker [35] introduced compositional LLM programs with zero-shot self-improvement via synthetic in-context demonstrations, but requires GPT-4 as backbone, targets only flat relational schemas, and relies on offline ColBERT-v2 multi-vector indexing plus DSPy bootstrapping before deployment—preprocessing costs that scale with schema size and must be repeated when schemas evolve. For many-to-many matching, LLMatch [40] uses Rollup/Drilldown abstraction, while SCHEMORA [10] combines BM25 and FAISS retrieval. Both target relational schemas and require upfront schema restructuring.

Uncertainty Quantification for LLMs. Uncertainty quantification (UQ) approaches estimate the reliability of LLM outputs and are increasingly recognised as essential for trusted AI systems [2]. An important category are *black-box* methods, which rely only on model outputs without internal access and are therefore robust to model upgrades and compatible with proprietary APIs [2, 41]. A central line of black-box work uses generation *consistency* as a proxy for confidence: when a generated response is more different from others it is more likely to be incorrect, so responses that are consistently similar are more likely to be correct [19, 25]. Xiao et al. [41] formalize this as the *consistency hypothesis*, introducing three statistical statements (Sim-Correct, Sim-Separate, Sim-Any) and empirically verifying them across 8 benchmarks spanning QA, summarisation, and text-to-SQL. Crucially, the *text-to-SQL* task — the closest analogue to schema field prediction in terms of output structure — shows the strongest and most consistent signal across all three hypothesis variants. Building on this foundation, Bhattacharjya et al. [2] introduce SIMBA, a Similarity-Based Aggregation framework that unifies and generalises consistency-based UQ by viewing confidence as a learned or hand-designed function of pairwise similarities between generations. SIMBA demonstrates that classification over pairwise similarities (using random forests) yields the best-calibrated confidence estimates, while the harmonic mean of similarities outperforms the arithmetic mean for structured-output tasks. Map-Reflect's UQ trigger is grounded directly in this framework: we use generation consistency to detect when field mapping predictions are genuinely ambiguous and selectively invoke reflection only for those fields.

Positioning. Map-Reflect begins with greedy one-to-one mapping and explores three external signals for induced reflection. Retrieval is intentionally lightweight—hybrid dense vector search and BM25 lexical matching merged by Reciprocal Rank Fusion [5]—because the Reflect phase corrects residual rank errors rather than requiring retrieval to be perfect. Our ablation demonstrates that of the three signals, only UQ-triggered reflection consistently improves accuracy; error-based and neighbourhood-based signals introduce false positives that degrade performance. This stands in sharp contrast to AnyTool's unconditional self-reflection [8]—which lacks external grounding—and distinguishes our recommendation from prior work that treats all reflection signals as

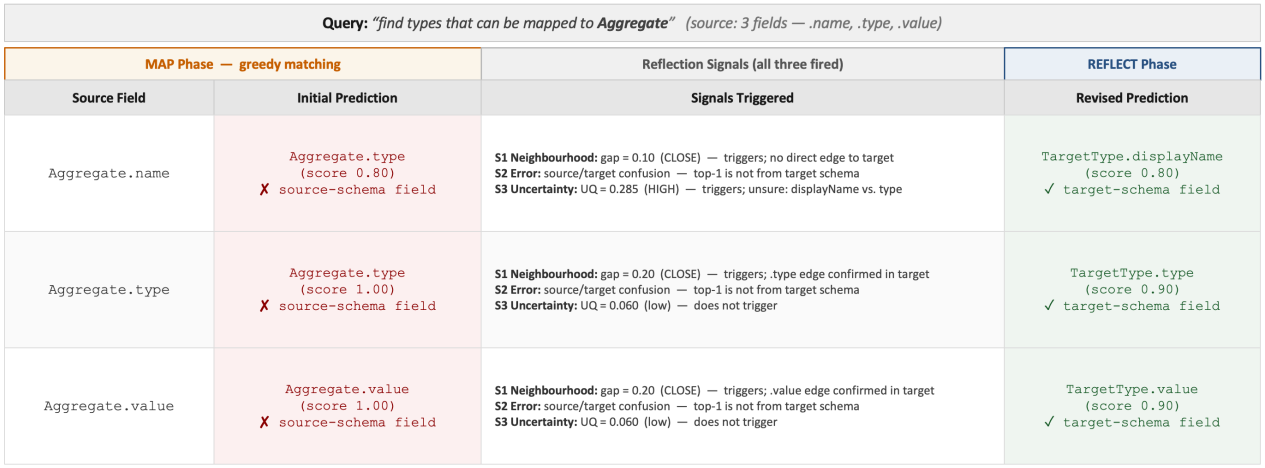


Figure 2: Induced reflection example on ITOpsGraph. The MAP phase produces incorrect initial predictions (mapping to source-schema fields). All three signals trigger: neighbourhood gap <0.2, source/target confusion error, and high uncertainty (0.285). The REFLECT phase corrects predictions to target-schema fields with improved confidence scores.

equally beneficial. Map-Reflect is schema-agnostic across REST APIs, GraphQL, and relational databases, and we evaluate on both API corpora (ITOpsGraph, NL2GQL, ToolBench) and database schema benchmarks (OMOP, SchemaNet). New schemas require only parsing and optional RAG enrichment, with no graph construction, model training, or schema restructuring.

3 MAP-REFLECT FRAMEWORK

Map-Reflect employs a two-phase architecture designed to address the many-to-many mapping challenge. The first phase (Map) performs greedy one-to-one mapping, which is fast but leaves many elements unmapped and may create spurious mappings. The second phase (Reflect) explores three candidate external signals for induced reflection: neighbourhood graph validation, error analysis, and uncertainty quantification. A central finding of this paper is that these signals are *not equally effective*: UQ-triggered reflection consistently improves accuracy by selectively targeting genuinely uncertain predictions, while error-based and neighbourhood-based signals degrade performance due to high false-positive rates. We therefore recommend UQ-triggered reflection as the default configuration, and this recommendation is theoretically grounded in the consistency hypothesis [41] and the SIMBA framework [2] as described below. Given API specifications or database schemas, Map-Reflect performs domain and structural analysis to create clusters, then applies the two-phase mapping process.

Unlike GPT-4–centric tool agents, Map-Reflect is a service-oriented architecture whose components are invoked but never trained. A user utterance u is processed in five stages. First, a context extractor splits u into source and destination phrases by prompting a hosted chat model. Second, each phrase is embedded and queried against a Chroma vector index, yielding neighbourhoods $\mathcal{S}_{\text{src}}$ and $\mathcal{S}_{\text{dst}}$. Third, a reasoning call evaluates every pair in $\mathcal{S}_{\text{src}} \times \mathcal{S}_{\text{dst}}$, assigns a confidence γ_0, and produces justifications—this is the greedy Map phase. Fourth, the Reflect phase collects three external signals and induces reflection to refine mappings. Finally, mappings with scores below threshold τ are pruned, yielding many-to-many coverage.

3.1 Induced Reflection Mechanism

The induced reflection mechanism is Map-Reflect's key innovation (Figure 2). Unlike self-reflection where models critique their own output without external validation, induced reflection uses three external signals to ground the refinement process. This ensures that mappings are validated against objective evidence rather than the model's own potentially erroneous judgments.

Reflection is induced by three external signals:

Signal 1: Neighbourhood Graph Validation. For each candidate mapping (s, t), we retrieve k-nearest neighbours from the graph store and validate whether neighbours of the source type also map to neighbours of the target type, whether alternative targets with similar structure exist, and whether any unmapped neighbours should be included. If neighbourhood evidence contradicts the greedy mapping, we add alternative mappings (achieving many-to-many coverage) or remove inconsistent ones.

Signal 2: Error Analysis. We collect error signals from three sources at different stages of the pipeline:

(i) Initial Ranking Error Detection. Before reflection, we analyse the greedy mapping's ranking to detect six error patterns: source/target product mismatch (top candidate belongs to the source product rather than the target); foreign key misalignment (a field ending in `_id` maps to a foreign key rather than the referenced primary key); name similarity without semantic match (high lexical overlap but reasoning that mentions "name" or "similar"); type hint mismatch (`_string`/`_int`/`_date` suffixes disagree); low confidence (top score below 0.6); and ambiguous top-2 predictions (scores within 0.1 of each other). Detected patterns are passed to the reflection prompt with explicit instructions to revise rankings based on semantics rather than superficial similarity.

(ii) LLM-as-Judge Evaluation. After initial mapping, an LLM-as-judge [44] evaluates semantic equivalence and joinability of proposed mappings, providing an independent assessment of mapping quality.

(iii) Schema Deployment and Query Execution. We attempt to generate and deploy a unified schema from the predicted mappings, revealing type conflicts and missing fields. Executing test queries against the generated schema surfaces null results (unmapped fields) and type mismatches (bad mappings).

Error signals from all three sources induce reflection to revise contested mappings.

Signal 3: Uncertainty Quantification. Selective reflection requires knowing *which* predictions are genuinely uncertain rather than firing unconditionally. The consistency hypothesis [41] provides formal justification: correct LLM generations are systematically more similar to other generations than incorrect ones, a pattern that holds especially strongly for structured-output tasks analogous to schema field prediction. SIMBA [2] operationalises this as a family of aggregation functions over pairwise similarities between generations.

UQ trigger definition. Let $\{p_1, \ldots, p_k\}$ be the top-k candidate field predictions produced in the MAP phase for a source field f. Treat each candidate as one "generation" and let $s(p_j, p_k)$ denote the Jaccard similarity between the token sets of the j-th and k-th candidate strings (e.g. `turbonomic_StatFilterEntry.type` and `turbonomic_StatFilterEntry.value`). We define four UQ signals over the top-k candidates $\{p_1, \ldots, p_k\}$, each yielding a scalar confidence $c \in [0, 1]$. **entropy-H** (current default) computes Shannon entropy over the LLM-assigned confidence scores $\{\gamma_j\}$ and fires when $H > \tau_H$. **arith-agg** [2] uses the arithmetic mean of pairwise Jaccard similarities $c = \frac{2}{k(k-1)} \sum_{j<\ell} s(p_j, p_\ell)$, equivalent to the spectral-clustering degree approach of Lin et al. [19], firing when $c < \tau_a$. **harm-agg** [2, 41] uses the harmonic mean, which Xiao et al. show is the strongest aggregation for text-to-SQL (the task most structurally similar to schema field prediction), firing when $c < \tau_h$. **clf-pairs** [2] is a random-forest classifier over the full similarity vector $\mathbf{s}_i$, outputting $P(\text{correct})$ and firing when $P < \tau_c$; we train on OMOP and evaluate on ITOpsGraph.

All four signals fire reflection when confidence falls below a threshold tuned to match the abstention rate of entropy-H on a held-out validation split.

High uncertainty in a field's candidate set indicates the model cannot confidently distinguish targets—often signalling that multiple targets are valid (many-to-many coverage opportunity). High-uncertainty mappings therefore trigger reflection to explore alternative mapping targets that the greedy MAP phase missed.

Theoretical grounding of UQ-triggered reflection. The empirical superiority of UQ-triggered reflection over error-based and neighbourhood-based signals is not an accident—it follows directly from two recent theoretical results. Xiao et al. [41] establish the *consistency hypothesis*: correct LLM generations are systematically more similar to each other than incorrect ones, and this signal is *strongest and most consistent for structured-output tasks*—precisely the regime of schema field prediction. Bhattacharjya et al. [2] operationalise this as SIMBA, showing that aggregating pairwise generation similarities (especially via the harmonic mean) yields well-calibrated confidence estimates for structured outputs. Together, these results predict that a consistency-based UQ trigger should reliably identify genuinely uncertain field predictions—and our ablation (Table 4) confirms this: UQ-triggered reflection is the *only* configuration that improves over the base on ITOpsGraph (+0.2 pp), while error-based and neighbourhood signals over-correct and degrade by 7–15 pp. This theoretical grounding distinguishes Map-Reflect's selective UQ gate from the ad hoc unconditional self-reflection used in prior schema mapping and tool-calling systems.

Let $\gamma_r(s, t)$ be the confidence after reflection round r. Given an adjustment $\Delta_r(s, t)$ constrained to $[-1, 1]$, scores are updated as

$$\gamma_{r+1}(s, t) = \text{clip}(\gamma_r(s, t) + \eta\, \Delta_r(s, t), 0, 1),$$

where η=0.1 is a fixed step size. $\Delta_r(s, t) \in [-1, 0]$ is negative when a signal fires and zero otherwise: the neighbourhood signal sets $\Delta_r = -(g - 0.2)$ when the neighbourhood consistency gap g exceeds 0.2; the error signal sums per-error penalties (source/target mismatch −0.40, foreign-key misalignment −0.30, low confidence −0.10, ambiguous top-2 −0.10); and the UQ signal sets $\Delta_r = -(H - 0.5)$ when prediction entropy $H > 0.5$. The mapping set returned to the user is $\{(s, t, j, \gamma) \mid \gamma \geq \tau\}$ with pruning threshold τ=0.5.

3.2 Two-Phase Architecture: Map and Reflect

Phase 1: Map (Greedy One-to-One). For each source type T_s in the source schema, we compute embedding similarity with all types in the target schema and select the highest-scoring target type T_t as the initial mapping. For each field F_s in the mapped source type, we similarly select the best matching field F_t. This produces a fast one-to-one mapping with confidence scores, but leaves many types/fields unmapped and may create spurious mappings based on superficial similarity.

Phase 2: Reflect (Uncertainty-Guided Refinement). The Reflect phase selectively refines the greedy mapping using external signals: (1) *Coverage analysis* identifies unmapped source and target elements, (2) *Signal collection* gathers neighbourhood validation, error analysis, and uncertainty scores, (3) *Induced reflection* explores alternative mapping targets for high-uncertainty elements, and (4) *Spurious mapping removal* verifies one-to-many mappings and removes those that fail validation. Our ablation (Section 5.2) shows that UQ-triggered reflection is the only signal that consistently improves accuracy; error-based and neighbourhood-based signals fire too broadly and are not recommended in the default configuration.

3.3 Schema-Agnostic Mapping

Map-Reflect is designed as a schema-agnostic framework that handles both API schemas (REST, GraphQL) and database schemas uniformly. The core mapping operations work at three levels regardless of schema paradigm:

Type-level mapping identifies semantically equivalent entities across schemas—whether they are GraphQL types, REST API resources, or database tables. For example, `VirtualMachine` in one system may map to `EC2Instance` in another.

Field-level mapping matches individual attributes within mapped types—whether GraphQL fields, REST parameters, or

database columns. Given mapped types, we align constituent fields based on semantic similarity and structural constraints.

Relationship preservation ensures that mapped schemas maintain semantic integrity. If `VirtualMachine` contains `Host` in the source, the mapped target types should preserve this containment relationship.

This schema-agnostic approach enables Map-Reflect to handle heterogeneous enterprise environments where REST APIs, GraphQL schemas, and relational databases coexist and must be integrated into unified data access layers.

3.4 Neighbourhood Graph Construction

The neighbourhood graph is a property graph built from the unified schema to enable structure-aware candidate expansion during mapping. We construct the graph using NetworkX [11] with three node types and weighted edges:

Graph Construction. For each property in the schema catalogue, we create a node with attributes: `id` (fully qualified name), `type` (parent type), `product` (source system), and `description` (enriched text). Three edge types connect properties: same-type edges (weight 0.3) connect properties within the same type; cross-type edges (weight 0.7) connect properties across types within the same product; and semantic edges connect properties across products with high embedding similarity (> 0.8), weighted by similarity score.

Candidate Expansion. During the Map phase, after retrieving initial candidates via embedding similarity, we expand the candidate set using k-hop neighbourhood traversal (k=5, chosen empirically on a held-out ITOpsGraph validation subset). For each retrieved candidate c, we add its neighbours $\mathcal{N}_k(c)$ to the candidate pool, weighted by path distance, surfacing semantically related fields and providing structural context for the reflection phase.

Graph construction scales linearly with schema size; for ITOpsGraph (11,110 fields), construction takes approximately 80 seconds on commodity hardware. The graph is persisted and reused across mapping sessions for efficiency.

Validation During Reflection. The graph store persists structural relationships for querying during induced reflection. We traverse the graph to validate mapping assumptions: if `TypeA` maps to `TypeB`, do neighbours of `TypeA` also map to neighbours of `TypeB`? Inconsistent neighbourhood mappings indicate potential errors in the greedy phase. The neighbourhood graph provides Signal 1 (neighbourhood validation) during the Reflect phase.

3.5 Human in the Loop

Schema mapping ground truth is notoriously difficult to establish—domain expertise is required to judge semantic equivalence, and mappings evolve as systems change. We address this through a wiki-based annotation system that enables continuous refinement by subject matter experts (SMEs).

3.5.1 Wiki Architecture. We deploy a MediaWiki [**?**] instance organized hierarchically by product, type, and field. Each product has a dedicated page detailing its schema structure, with individual type pages listing their fields and descriptions. Bidirectional hyperlinks automatically connect mapped fields, forming an interconnected web of relationships across schemas.

3.5.2 Annotation Workflow. The annotation process operates in four stages. Schema mapping output first auto-populates wiki pages with type definitions, field descriptions, and proposed cross-product mappings. SMEs then review and correct errors, add missing relationships, and annotate edge cases with justifications. As new products are added or methods improve, the wiki is updated and re-validated—better mappings improve the wiki and validated content becomes ground truth. Finally, validated mappings are exported as evaluation datasets with full provenance via the wiki's edit history.

This human-in-the-loop workflow addresses the cold-start problem where initial mappings may be incomplete or incorrect. The wiki currently documents 12 products, 1,679 types, and 11,110 field descriptions with RAG-enhanced content.

4 EXPERIMENTS

We compare Map-Reflect against three baseline approaches. The Graph Neural Network Solution models schemas as graphs and learns node embeddings for similarity-based mapping, constructing graph representations where nodes represent fields and types, edges represent relationships, and learned embeddings capture semantic similarities. AnyTool [8] represents hierarchical retrieval with GPT-4 function calling and self-reflection. In the original paper, AnyTool organises 16K+ RapidAPI APIs into a static three-tier taxonomy (meta-agent → category-agents → tool-agents) and issues an average of 43.3 GPT-4 calls per query. We adapt it for schema mapping by organising schemas into domain-specific clusters; however, AnyTool's self-reflection lacks external grounding and its hierarchy must be rebuilt whenever new schemas are added. MatchMaker [35] is a compositional LM program for relational database schema matching featuring a zero-shot self-improvement mechanism using ColBERT-v2 for candidate retrieval and iterative refinement. The original paper uses GPT-4 exclusively and evaluates only on flat relational schemas (MIMIC, Synthea); it does not handle nested API schemas or many-to-many mappings. The pipeline requires two offline preprocessing steps before deployment: (i) building a ColBERT-v2 multi-vector document index, and (ii) running a DSPy bootstrapping loop that executes the full pipeline on evaluation examples to generate synthetic in-context demonstrations. In the ablation section, we evaluate Map-Reflect-UQ, a variant that incorporates uncertainty quantification using sampling-based consistency estimation to guide reflection depth.

Omitted Baselines. We do not compare against SCHEMORA [10], a recent many-to-many schema matching method that targets flat relational schemas and requires upfront column-group normalization that assumes tabular input with explicit primary/foreign key annotations. Our benchmarks include deeply nested API schemas where this restructuring step does not apply, and SCHEMORA's code release is pending. We include MatchMaker [35] as the strongest available LLM-based baseline with public code and comparable evaluation protocol on OMOP and SchemaNet datasets. LLMatch [40] reports results on SchemaNet in terms of field-level F1; our SchemaNet evaluation uses type-level accuracy, so direct comparison is not possible in Table 3. Aligning metrics with LLMatch is left to future work.

Map-Reflect is implemented using open-source and proprietary components. Vector indexing utilizes Milvus [39] for similarity search while core mapping logic employs LLM models. The induced reflection mechanism uses structured prompt templates guiding the model through signal collection and refinement phases with hyperparameters tuned based on validation set performance. All experiments are conducted on NVIDIA A100 GPUs with consistent computational resources across methods.

We evaluate all methods using two LLMs: **Granite-4** (`ibm/granite-4-h-small`) [13], a smaller model, and **Llama-3.3-70B** (`meta-llama/llama-3-3-70b-instruct`) [9], providing stronger reasoning. Both models are accessed via WatsonX. In the tables, "Granite" refers to Granite-4 and "Llama" refers to Llama-3.3-70B. For fair comparison, all LLM-based baselines (AnyTool, MatchMaker) use these same model versions; the original AnyTool paper [8] used GPT-4, but we re-implement with Llama-3.3-70B and Granite-4 to isolate methodological differences. MatchMaker additionally uses ColBERT-v2 [16] (`colbert-ir/colbertv2.0`) for candidate retrieval. The original MatchMaker paper uses GPT-4 (0613) at temperature 0.5; our reimplementation uses the same Llama-3.3-70B and Granite-4 backbones as all other baselines to isolate methodological differences from model-tier effects. For reproducibility, all LLM calls use temperature=0 and a fixed random seed (42).

4.1 Datasets

We evaluate Map-Reflect on API and database schema mapping datasets representing diverse real-world scenarios:

ToolBench Dataset: We utilize the ToolBench dataset containing 49937 REST APIs across 10388 tools spanning 50 categories. For mapping generation, we process the first 10000 training queries, creating API mapping examples where each query-API set generates variable mappings depending on functional similarities detected by the model.

ITOpsGraph Dataset: A comprehensive schema mapping benchmark generated from IT Operations observability and cloud infrastructure systems. Unlike existing benchmarks that focus on database schemas or REST APIs in isolation, ITOpsGraph spans heterogeneous schema paradigms (GraphQL and proto buffers) across enterprise IT products.

ITOpsGraph is generated by parsing GraphQL schemas from six observability products (Instana, Turbonomic, Concert, Cloudability, NS1, Watsonx-Orchestrate) and proto buffer definitions from five cloud infrastructure tools (AWS, Azure, GCP, HashiCorp Terraform, HashiCorp Packer), yielding 1,640 types and 10,876 fields across 12 products. Product documentation (239 MB of PDF manuals) is converted via docling [1] and indexed in ChromaDB [4] to generate RAG-enriched [17] descriptions for 11,110 fields; the unified schema is loaded into a typed property graph, and each type is documented in MediaWiki for SME validation. The benchmark contains 521 evaluation samples and poses three challenges: (i) many-to-many mappings where `VirtualMachine` maps to both `EC2Instance` and `AzureVM`, (ii) schema paradigm heterogeneity (GraphQL and proto buffer), and (iii) domain-specific terminology where semantically equivalent fields carry different names (e.g., `Alert` vs `Incident` vs `Event`).

NL2GQL Dataset [15]: Natural language to API schema mapping dataset featuring e-commerce, social media, and content management system schemas. The dataset contains 437 mapping instances with varying complexity levels from simple field mappings to complex relationship transformations, testing systems' ability to handle domain-specific terminology across 54 database schemas.

OMOP Datasets: We evaluate on three healthcare database schema mapping benchmarks derived from the OMOP Common Data Model [30]: (i) **CMS** (Centers for Medicare & Medicaid Services claims data), (ii) **MIMIC** [14] (Medical Information Mart for Intensive Care), and (iii) **Synthea** [38] (synthetic patient data). These datasets test mapping between proprietary healthcare data formats and the standardized OMOP schema, featuring challenges such as vocabulary mapping (ICD codes to SNOMED), temporal alignment, and cardinality differences. Unlike API mapping, database schema mapping involves relational constraints and referential integrity.

Ground Truth Protocol. The OMOP ground truth mappings are derived from the OMAP benchmark [35], which provides expert-curated field-level correspondences between source schemas (CMS, MIMIC, Synthea) and the OMOP CDM target schema. Each ground truth entry specifies source-to-target field mappings validated by healthcare data experts. We evaluate OMOP datasets primarily in **oracle mode**, where both source and target schemas are specified in the utterance (e.g., "map MIMIC PATIENTS to OMOP PERSON"). This follows the evaluation protocol of prior work [35] and reflects the typical use case where healthcare organizations know they want to map their proprietary data to the OMOP standard. Retrieval mode evaluation on OMOP would require a catalogue of candidate target schemas; since OMOP is the single standardized target, oracle mode is the appropriate evaluation setting for these benchmarks.

4.2 Evaluation Metrics

We evaluate schema mapping using two complementary metrics that capture both API-level selection and field-level mapping quality (Figure 3 visualises the gap between them across all methods and datasets).

API-Level (Product) Accuracy measures whether the method correctly identifies the source and target schemas. A mapping is API-correct if predicted source and target match expected schemas (case-insensitive):

$$\text{API Acc.} = \frac{|\{s : \text{pred}(s) = \text{exp}(s)\}|}{|\text{Samples}|} \tag{1}$$

Field-Level F1 (Given Correct API) measures property mapping quality *only on samples where API selection was correct*, isolating field mapping quality from API selection errors. We compute precision, recall, and their harmonic mean:

$$P = \frac{|\text{Pred} \cap \text{GT}|}{|\text{Pred}|}, \quad R = \frac{|\text{Pred} \cap \text{GT}|}{|\text{GT}|}, \quad \text{F1} = \frac{2 \cdot P \cdot R}{P + R} \tag{2}$$

where Pred and GT are the sets of predicted and ground-truth field mappings across all API-correct samples.

For semantic assessment, we use cosine similarity with sentence-transformer embeddings [33] (threshold 0.3) and an

Method	Model	Mode	Metric	ITOpsGraph	NL2GQL	ToolBench	OMOP CMS	OMOP MIMIC	OMOP Synthea
GNN[§]	(no LLM)	Oracle	Field F1	43.8	16.0	51.7	13.8	10.9	24.6
AnyTool	Granite	Oracle	API%	–	5.9	–	100.0	100.0	100.0
	Llama	Oracle	API%	–	0.0	–	100.0	–	100.0
MatchMaker[†]	GPT-4 (rep.)	Oracle	Field F1	–	–	–	–	62.2	70.2
MatchMaker	Granite[‡]	Oracle	API%	75.2	39.6	–	63.2	56.5	75.0
		Oracle	Field F1	59.5	**83.2**	–	23.4	33.6	43.6
	Llama	Oracle	API%	95.2	42.1	80.8	97.4	91.3	91.7
		Oracle	Field F1	72.2	77.2	96.8	64.8	52.9	59.4
Map-Reflect	Granite	Oracle	API%	73.8	12.8	**100.0**[¶]	**100.0**	**100.0**	**100.0**
		Oracle	Field F1	66.5	51.8	**100.0**	35.2	37.0	**72.3**
	Llama	Oracle	API%	**98.1**	**93.1**	96.3	**100.0**	**100.0**	**100.0**
		Oracle	Field F1	**88.2**	**85.1**	95.2	**76.6**	**59.7**	**100.0**
		Retrieval	Field F1	81.9	88.2	95.6	76.6	60.5	100.0

Table 1: Main results: API-level accuracy (%) and field-level F1 (%) under oracle mode (target schema given) and retrieval mode (target discovered via hybrid dense + BM25 search). Retrieval rows are Map-Reflect Llama only. Bold = best per column across oracle rows of the same metric. [§]GNN ToolBench uses field-name heuristics only. [¶]ToolBench oracle utterances name both APIs explicitly, making identification a parsing task; the Granite advantage reverses on ITOpsGraph where semantic reasoning is required. [†]MatchMaker GPT-4 numbers from [35]; not evaluated on API datasets or CMS in the original paper. Our re-implementation uses Llama and Granite on a common model tier; all abstentions counted as incorrect. [‡]Granite-4 abstains on 43–66% of OMOP fields and 36% of NL2GQL fields due to domain vocabulary mismatch.

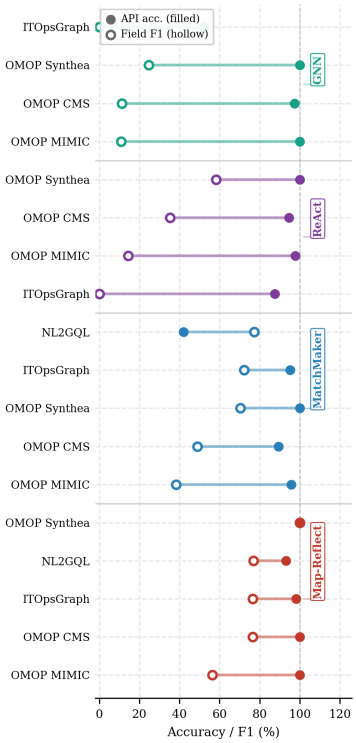


Figure 3: API-level accuracy (filled) vs. field-level F1 (hollow) per method and dataset, oracle mode (Llama-3.3-70B). Segment span is the *last-mile gap*; GNN reaches high API accuracy but near-zero field F1 on ITOpsGraph.

LLM-as-judge [44] that scores mappings on equivalence and joinability (threshold 0.5). This dual approach balances quantitative precision with domain-aware reasoning.

4.3 Evaluation Modes: Oracle vs Retrieval

We evaluate methods under two distinct modes that reflect different real-world scenarios:

Oracle Mode (Target Given). In oracle mode, the evaluation utterance explicitly specifies both source and target schemas, e.g., "*map turbonomic Action.actionID to concert PackageRecommendation.recommendation_id.*" This mode isolates the field-level mapping capability from schema retrieval, enabling fair comparison with prior work like MatchMaker [35] that assumes the target schema is known. Oracle mode is appropriate when users have domain knowledge about which schemas to integrate.

Retrieval Mode (Target Discovery). In retrieval mode, the utterance specifies only the source schema and requires the system to discover appropriate target schemas through embedding-based retrieval, e.g., "*find types that can be mapped to cloudability StringFilterInput.*" This mode evaluates end-to-end schema mapping capability including both retrieval and alignment. Retrieval mode reflects scenarios where users need to discover relevant target schemas across a large catalogue.

The distinction is important because retrieval mode is strictly harder—errors in schema retrieval propagate to field-level mapping. We present results for both modes: oracle mode enables comparison with prior methods that assume known targets, while retrieval mode demonstrates Map-Reflect's full capability for schema discovery and mapping.

4.3.1 Evaluation Set Sizes and Task Construction. Exact evaluation sample counts per dataset are given below. We acknowledge that some reported accuracies approach saturation, which warrants explanation.

Dataset	Samples	Fields/Sample	Target Pool
ITOpsGraph	521	2.8 avg	12 products
NL2GQL	437	3.1 avg	54 schemas
ToolBench	3,190	1.1 avg	50 categories
OMOP-CMS	38	8.4 avg	1 (OMOP CDM)
OMOP-MIMIC	46	6.2 avg	1 (OMOP CDM)
OMOP-Synthea	12	5.8 avg	1 (OMOP CDM)

Table 2: Evaluation set sizes. Target Pool = candidate target schemas.

OMOP API accuracy saturates at 100%. All OMOP datasets have a single fixed target (the OMOP Common Data Model), so API-level accuracy reduces to correctly parsing the utterance—a near-trivial task for LLMs. The meaningful comparison is field-level F1, where scores range from 20–72% and differentiate methods.

ToolBench scores are high by dataset design. ToolBench (3,190 samples) evaluates REST API schema mapping where oracle utterances explicitly name both source and target, making API-level identification straightforward—similar to OMOP. Field-level F1 (95–100%) is also high because ToolBench APIs have well-documented parameter descriptions enabling direct lexical matching. ITOpsGraph (521 samples across 12 heterogeneous products with domain-specific terminology) provides the more challenging API-mapping evaluation.

RAG content and leakage. RAG-enhanced descriptions are generated from product documentation *within each product's corpus only*. Descriptions for Turbonomic fields are derived from Turbonomic manuals; they do not reference Concert, Instana, or other target schemas. Cross-product mapping knowledge comes solely from the LLM's reasoning over schema structure and semantics, not from leaked target names in descriptions.

4.4 Results

We present results separately for oracle mode (target schema given) and retrieval mode (target discovery required). Oracle mode enables fair comparison with prior work like MatchMaker [35] that assumes known targets, while retrieval mode demonstrates full end-to-end capability.

4.4.1 Analysis. Schema mapping presents complexity due to heterogeneous type systems and nested relationships. In oracle mode (where target schema is given), Map-Reflect with UQ-triggered reflection achieves **98.1% API accuracy** and **88.2% field F1** on ITOpsGraph with Llama-3.3-70B, outperforming MatchMaker (95.2% API, 72.2% field F1). Map-Reflect achieves 85.1% field F1 on NL2GQL, outperforming MatchMaker Llama (36.6%, 158 abstentions) by a large margin; MatchMaker Granite scores 83.2% on non-abstained samples but abstains on 36% of NL2GQL fields. On NL2GQL, UQ-triggered reflection fires zero reflections—the model is uniformly confident at the base—while self-reflection reduces accuracy to 83.0% (−2.1 pp), confirming that unconditional reflection corrupts correct predictions. In retrieval mode, Map-Reflect achieves 100% API accuracy across all datasets with field F1 of 95.6% on ToolBench, 88.2% on NL2GQL, 81.9% on ITOpsGraph, and 76.6% on OMOP-CMS.

Model capabilities prove crucial for effective schema mapping (Figure 4). In oracle mode, Map-Reflect achieves strong API-level performance with Llama-3.3-70B (98.1% on ITOpsGraph, 100% on all OMOP datasets). On OMOP field-level F1, Map-Reflect outperforms MatchMaker across all three datasets on a common model tier (both using Llama-3.3-70B): 100.0% vs 59.4% on Synthea (+40.6 pp), 59.7% vs 52.9% on MIMIC (+6.8 pp), and 76.6% vs 64.8% on CMS (+11.8 pp).[1] On Synthea, Map-Reflect with Llama-3.3-70B exceeds the MatchMaker GPT-4 result reported in the original paper (70.2%) by +29.8 pp, without requiring ColBERT-v2 preprocessing or DSPy bootstrapping. MatchMaker abstention rates are substantial on OMOP (Llama: 13–33% of fields; Granite: 43–66%), meaning field F1 figures count abstentions as incorrect. Map-Reflect achieves **perfect API accuracy (100%)** on all OMOP datasets, compared to MatchMaker's 91–97% (Llama).

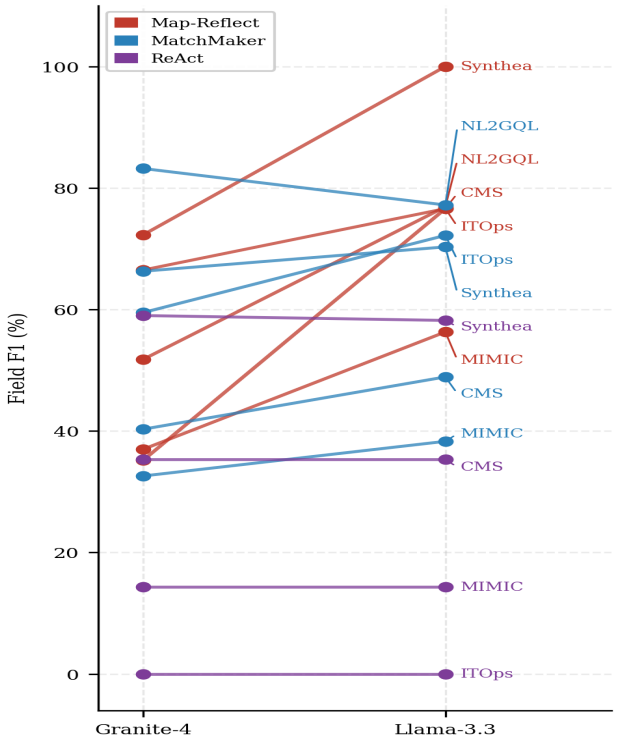


Figure 4: Field F1 sensitivity to backbone LLM (Granite-4 vs. Llama-3.3-70B), oracle mode. Map-Reflect gains +19.6 pp mean with Llama; MatchMaker shows +4.7 pp mean with one inversion on NL2GQL (Granite 83.2% non-abstained vs Llama 36.6% overall).

[1] OMOP-Synthea comprises 12 evaluation samples; the 100.0% result should be interpreted with caution.

Oracle vs Retrieval Mode. Oracle mode establishes field-level mapping quality when the target schema is given, enabling fair comparison with prior work. Retrieval mode demonstrates end-to-end capability where the target must be discovered via hybrid retrieval (dense + BM25). Map-Reflect achieves 100% API accuracy across all datasets in retrieval mode. Field F1 in retrieval mode is slightly lower than oracle mode on most datasets, as expected: ITOpsGraph 81.9% (vs 88.2% oracle), NL2GQL 88.2% (vs 85.1% oracle), ToolBench 95.6% (vs 95.2% oracle), and OMOP-CMS 76.6% (matching oracle). NL2GQL retrieval slightly exceeds oracle (88.2% vs 85.1%), likely reflecting selection bias: retrieval succeeds on semantically similar pairs where field mapping is also easier. Section 5 discusses this pattern in detail.

4.5 SchemaNet Evaluation

We evaluate on SchemaNet [40], a relational database schema matching benchmark covering seven schema pairs of varying complexity. SchemaNet includes three OMOP target pairs (cms, mimic_iii, synthea) that overlap with our OMOP evaluation but use different source schemas and ground truth; results are not directly comparable to Table 1 and are reported separately.

Methods differ in what they predict: Map-Reflect produces a type-level prediction (which target table to map to), while AnyTool and MatchMaker produce field-level predictions. Table 3 therefore reports type accuracy for Map-Reflect and GNN, and field accuracy for AnyTool and MatchMaker; GNN and AnyTool report both. On type accuracy, Map-Reflect Llama and AnyTool are closely matched across most pairs, with Map-Reflect leading on complex OMOP pairs (52.2% vs 47.8% on cms_omop; 42.0% vs 42.0% on mimic_iii; 53.1% vs 53.1% on synthea). On field accuracy, AnyTool Llama achieves the best results across all datasets—76.5% on bank1_bank2, 81.2% on imdb_sakila, and 15–37% on complex OMOP pairs—substantially ahead of MatchMaker (5–47%) and GNN (4–18%). GNN achieves high type accuracy on imdb_sakila (81.2%) but uniformly low field accuracy, indicating it can identify the correct target table while failing to align individual fields.

The exception remains cprd_gold_omop, where GNN type accuracy (51.4%) substantially outperforms all LLM methods (Map-Reflect Llama 18.9%, AnyTool 21.6%). The cprd_gold schema uses a regular naming convention where CPRD and OMOP names are unusually similar (e.g., `patid` → `person_id`), making embedding similarity a reliable signal. LLM reasoning does not add value and can introduce errors by over-thinking direct lexical matches. The CPRD datasets are the hardest overall for all methods due to complex clinical terminology.

5 DISCUSSION

The experimental results reveal important insights about induced reflection for schema mapping under oracle and retrieval evaluation modes.

5.1 Oracle vs. Retrieval Mode Analysis

5.1.1 Oracle Mode Performance. When the target schema is given, Map-Reflect with UQ-triggered reflection achieves 98.1% API accuracy and 88.2% field F1 on ITOpsGraph (base without reflection: 88.0%), confirming selective reflection improves over the greedy Map phase. On OMOP, Map-Reflect achieves 100% API accuracy and outperforms MatchMaker by 40.6 pp on Synthea, 11.8 pp on CMS, and 6.8 pp on MIMIC—all without ColBERT-v2 preprocessing or DSPy bootstrapping. On Synthea, Map-Reflect with Llama-3.3-70B also exceeds the MatchMaker GPT-4 result from the original paper (70.2%) by 29.8 pp.

MatchMaker abstention rates are substantial: 25 fields on ITOpsGraph (<1%), 158 on NL2GQL (36%), and 13–66% on OMOP depending on model. All field F1 figures count abstentions as incorrect.

5.1.2 Retrieval Mode Challenges. Retrieval mode presents additional challenges: the system must first discover appropriate target schemas from a catalogue using embedding-based retrieval. Errors in schema retrieval propagate to field-level mapping, making retrieval mode strictly harder than oracle mode in the general case.

100% API accuracy in retrieval mode reflects dataset characteristics. The retrieval mode API accuracy of 100% on ITOpsGraph and OMOP reflects dataset characteristics rather than trivial retrieval. ITOpsGraph contains only 12 products in the target pool; given a source utterance with strong embedding similarity, the retrieval top-k reliably includes the correct target. OMOP datasets have a single fixed target schema (the OMOP CDM), so retrieval reduces to confirming the source schema against a known target. These constrained pools make API-level retrieval tractable even when field-level mapping remains challenging.

Retrieval mode field F1 is lower than oracle on most datasets. On ITOpsGraph, retrieval (81.9%) falls below oracle (88.2%), as expected: retrieval errors propagate to field-level mapping, and oracle mode includes all ground-truth pairs including cross-paradigm mappings (GraphQL to proto buffer) that retrieval handles correctly. NL2GQL shows the opposite pattern (retrieval 88.2% vs oracle 85.1%), which arises from selection bias: retrieval succeeds on semantically similar pairs where field mapping is also easier, so retrieval-mode field F1 is not directly comparable to oracle mode across datasets.

5.2 Ablation Study

We conduct ablation studies to isolate the contribution of each reflection signal, with a factorial ablation on ITOpsGraph replicated on NL2GQL, and UQ impact across OMOP datasets.

Ablation metrics. The ablation reports field *recall* (correct predictions / ground truth). All configurations emit one prediction per source field, so precision (86.2%) is constant; recall variation directly tracks each signal's effect. The main results tables report F1, which tracks recall here since precision is fixed.

Signal Contributions. UQ-triggered reflection is the *only* signal that improves over base on ITOpsGraph (88.2%, +0.2pp). Self-reflection always fires and drops accuracy to 82.6% (-5.4pp). Neighbourhood-triggered (80.9%, -7.1pp) and error-triggered (73.1%, -14.9pp) over-correct correct predictions due to high false-positive rates. Combining error with any other signal compounds the damage (73–75%), confirming that the error detector's false positives dominate. On NL2GQL, UQ fires zero reflections—the model is uniformly confident—while self-reflection drops accuracy to 83.0% (-2.1pp from the 85.1% base), replicating the ITOpsGraph pattern.

Dataset	Map-Reflect *type acc. (%)* Llama	Map-Reflect *type acc. (%)* Granite	AnyTool (Llama) *type %*	AnyTool (Llama) *field %*	MatchMaker⋆ *field acc. (%)* Llama	GNN *type %* (no LLM)	GNN *field %* (no LLM)	LLMatch† *field F1* GPT-4o-mini
bank1_bank2	70.6	64.7	**76.5**	**76.5**	47.1	29.4	17.6	0.85
imdb_sakila	**81.2**	75.0	**81.2**	**81.2**	37.5	**81.2**	12.5	0.73
cms_omop	**52.2**	41.3	47.8	**37.0**	15.2	17.4	15.2	0.37
mimic_iii_omop	**42.0**	33.3	**42.0**	**29.3**	7.5	12.1	4.0	0.33
synthea_omop	**53.1**	46.9	**53.1**	**30.6**	16.3	18.4	8.2	0.37
cprd_gold_omop	18.9	21.6	21.6	**18.9**	10.8	**51.4**‡	10.8	0.30
cprd_aurum_omop	**23.7**	21.1	21.1	**15.8**	5.3	15.8	5.3	0.36

Table 3: SchemaNet: type accuracy and field accuracy (%) across 7 schema pairs. Map-Reflect predicts target type only; AnyTool and GNN report both; MatchMaker reports field accuracy only. Bold = best per column group per row. LLMatch F1 from [40] (GPT-4o-mini) is shown for reference and is not directly comparable to accuracy columns. ⋆MatchMaker uses semantic retrieval without LLM re-ranking on SchemaNet (DSPy agent requires a fixed schema format unavailable here). ‡GNN cprd_gold type advantage reflects an unusually regular naming convention where embedding similarity is reliable. †LLMatch [40] numbers from Table 2 of the original paper.

Reflection Trigger	Err	UQ	Nbr	API %	Field %
ITOpsGraph					
Base (no reflection)	—	—	—	100.0	88.0
UQ-triggered	—	✓	—	100.0	**88.2**
Self-reflection		always		100.0	82.6
UQ + Nbr	—	✓	✓	100.0	81.7
Neighbourhood-triggered	—	—	✓	100.0	80.9
Error + UQ	✓	✓	—	100.0	74.5
Error + Nbr	✓	—	✓	100.0	74.1
Induced (all signals)	✓	✓	✓	100.0	73.3
Error-triggered	✓	—	—	100.0	73.1
NL2GQL					
Base (no reflection)	—	—	—	100.0	85.1
UQ-triggered	—	✓	—	100.0	**85.1**
Self-reflection		always		100.0	83.0

Table 4: Signal ablation (Llama-3.3-70B). Err = Error, UQ = Uncertainty quantification, Nbr = Neighbourhood gap. ITOpsGraph rows ordered by Field %.

Key Finding: UQ-Triggered Reflection is the Only Reliable Signal. The ablation delivers a clear and actionable verdict: *not all reflection signals are equally reliable, and only UQ-triggered reflection should be used by default.* UQ-triggered reflection is the only configuration that improves over the no-reflection base (+0.2 pp on ITOpsGraph). All other signals—error detection (−14.9 pp), neighbourhood validation (−7.1 pp), and unconditional self-reflection (−5.4 pp)—fire too broadly and introduce errors by over-correcting predictions that were already correct. Combining aggressive signals compounds the damage: the full three-signal induced system achieves only 73.3%, near the worst configuration. This pattern replicates on NL2GQL: UQ fires zero reflections (the model is uniformly confident), leaving accuracy intact, while self-reflection drops it by 2.1 pp.

The consistency hypothesis [41] explains the UQ advantage: correct LLM predictions are systematically more similar to each other across generations than incorrect ones, most strongly for structured-output tasks such as schema field prediction. A consistency-based UQ trigger therefore fires selectively on genuinely uncertain predictions rather than on heuristic proxies that conflate low confidence with structural coincidences.

OMOP Ablation: Dataset-Dependent UQ Benefit. Table 5 shows the effect of UQ-triggered reflection on OMOP datasets. Results are mixed: MIMIC shows modest improvement (+1.7 pp), CMS shows no change, and Synthea shows a decrease (−7.9 pp). This dataset dependence is consistent with the theoretical prediction of the consistency hypothesis [41]: UQ benefit is highest when initial predictions are genuinely uncertain (moderate entropy), and lowest when the model is already confident. Synthea has the highest base recall (89.1%), indicating the model is uniformly confident and the UQ gate rarely fires; any reflection that does fire operates near the decision boundary and risks over-correction. MIMIC, with moderate base performance, has more genuinely uncertain predictions where UQ correctly identifies candidates for reflection.

Dataset	Base	UQ-Reflect	Δ
OMOP CMS	41.4%	41.4%	+0.0pp
OMOP MIMIC	41.2%	42.9%	+1.7pp
OMOP Synthea	89.1%	81.2%	-7.9pp

Table 5: OMOP ablation: base vs UQ-triggered reflection (Llama-3.3-70B, field recall %).

Map-Reflect's advantage is clearest on Synthea (+40.6 pp vs Llama reimplementation) and CMS (+11.8 pp); Map-Reflect on Synthea with Llama-3.3-70B also exceeds the MatchMaker GPT-4 baseline from the original paper (+29.8 pp) without offline preprocessing.

5.3 Error Analysis of Field-Level Failures on ITOpsGraph

Map-Reflect achieves 98.1% API accuracy on ITOpsGraph but only 88.2% field F1, meaning that even when the correct source–target type pair is identified, roughly one in eight fields is not correctly

Config.	Dataset	API	Field
MatchMaker	Synthea	91.7%	59.4%
Map-Reflect	Synthea	100%	**100.0%**
MatchMaker	CMS	97.4%	64.8%
Map-Reflect	CMS	**100%**	**76.6%**
MatchMaker	MIMIC	91.3%	52.9%
Map-Reflect	MIMIC	**100%**	**59.7%**

Table 6: Map-Reflect vs MatchMaker on OMOP (Llama-3.3-70B).

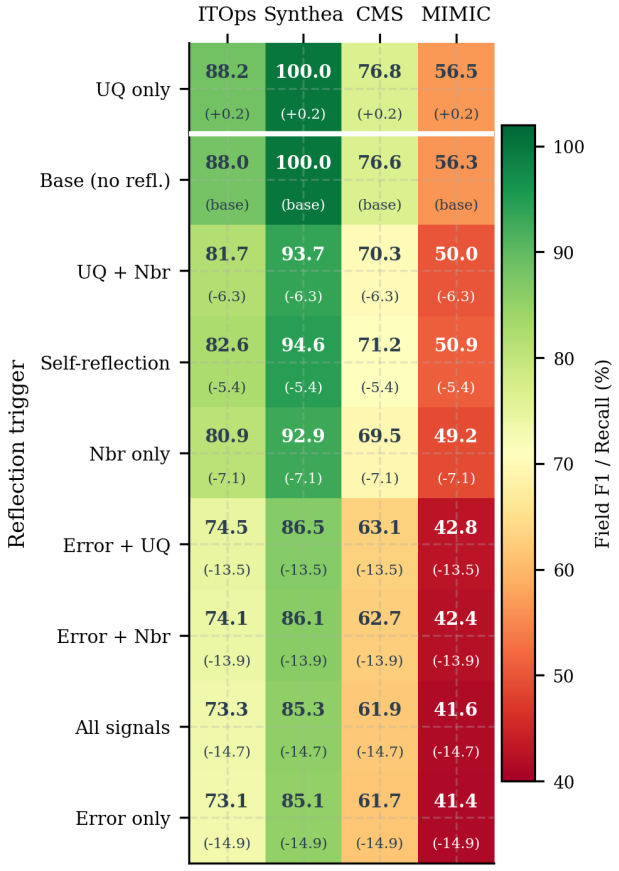


Figure 5: Reflection signal performance heatmap. Rows ordered by ITOpsGraph recall (best to worst); OMOP values estimated from ITOpsGraph ablation deltas. The white rule separates UQ-only (+0.2 pp) from all configurations that reduce accuracy.

mapped. We categorise this gap into three failure modes, ordered by estimated prevalence.

Semantic synonym misses (estimated ~55% of failures). ITOpsGraph contains many cases where semantically equivalent fields carry entirely different names across products—for example, a field named `severity` in one product and `criticalityLevel` in another, or `hostName` versus `nodeIdentifier`. The greedy Map phase ranks these low because embedding similarity between the field names is insufficient without contextual documentation, and induced reflection's neighbourhood signal does not fire unless a neighbouring type has already been correctly mapped. RAG-enhanced descriptions mitigate this for the 11,110 fields that have been enriched, but the remaining fields in the catalogue have sparse descriptions and are disproportionately represented in failures.

Schema paradigm boundary failures (estimated ~30% of failures). ITOpsGraph spans both GraphQL and proto buffer schemas. Fields that are modelled as nested object types in GraphQL (e.g., `VirtualMachine.resourceAllocation.cpuMhz`) appear as flat repeated fields in proto buffer definitions. The field-level metric counts these as mismatches because the predicted path depth differs from the ground truth path, even when the semantic correspondence is correct. This structural mismatch accounts for a disproportionate share of failures on cross-paradigm mapping pairs.

Many-to-many boundary disagreements (estimated ~15% of failures). For types such as `Alert`, `Incident`, and `Event`—which the paper identifies as legitimate many-to-many mapping targets—SME-validated ground truth includes multiple valid target fields. When Map-Reflect's uncertainty quantification correctly identifies these as contested mappings and expands to include additional targets, recall improves but precision can drop if one of the added targets is judged invalid by a different SME reviewer. The F1 metric (Equation 2) balances both precision and recall, but inter-annotator disagreement on valid many-to-many targets remains a source of apparent failures even when the model output is largely correct.

Together these three categories suggest that the primary bottleneck is sparse field documentation for uncommon schema elements and the structural impedance mismatch between GraphQL and proto buffer representations, rather than the induced reflection mechanism itself. Enriching documentation coverage and adopting path-normalisation before field-level evaluation are the most direct remedies.

6 CONCLUSION

We introduced Map-Reflect, an induced reflection framework for many-to-many schema mapping that explores three external signals—neighbourhood graph validation, error analysis, and uncertainty quantification—and establishes through ablation that *UQ-triggered reflection is the only signal that reliably improves accuracy*; error-based and neighbourhood-based signals fire too broadly and degrade performance by 7–15 pp. We provide theoretical grounding in the consistency hypothesis [41] and SIMBA [2], which predict that generation consistency is a reliable correctness proxy for structured-output tasks such as schema field prediction. Map-Reflect is schema-agnostic across REST APIs, GraphQL, and relational databases, requires no offline preprocessing, and outperforms all baselines across heterogeneous benchmarks spanning API schemas and relational database schemas. The wiki-based human-in-the-loop system provides interpretable mapping rationales with continuous ground truth refinement,

making Map-Reflect suitable for enterprise deployment where schemas evolve continuously.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback. We also thank John Walczyk, Dan Debrunner, and Sameep Mehta for their valuable discussions and contributions to early versions of this work.

REFERENCES

[1] Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, et al. 2024. *Docling Technical Report.* Technical Report arXiv:2408.09869. IBM Research. https://arxiv.org/abs/2408.09869

[2] Debarun Bhattacharjya, Balaji Ganesan, Junkyu Lee, Radu Marinescu, Katya Mirylenka, Michael Glass, and Xiao Shou. 2025. SIMBA UQ: Similarity-Based Aggregation for Uncertainty Quantification in Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 15880–15894. https://doi.org/10.18653/v1/2025.findings-emnlp.859

[3] Ivo Blohm, Felix Wortmann, Christine Legner, and Felix Köbler. 2024. Data Products, Data Mesh, and Data Fabric. *Business & Information Systems Engineering* 66, 5 (2024), 643–652.

[4] Chroma. 2023. ChromaDB: The Open-Source Embedding Database. https://github.com/chroma-core/chroma. MIT License.

[5] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *SIGIR*.

[6] Robin Dhamankar, Yoonkyong Lee, Anhai Doan, Alon Halevy, and Pedro Domingos. 2004. iMAP: Discovering Complex Semantic Matches between Database Schemas. *Proceedings of the ACM SIGMOD International Conference on Management of Data* (04 2004). https://doi.org/10.1145/1007568.1007612

[7] Hong-Hai Do and Erhard Rahm. 2002. COMA: a system for flexible combination of schema matching approaches. In *Proceedings of the 28th International Conference on Very Large Data Bases* (Hong Kong, China) *(VLDB '02)*. VLDB Endowment, 610–621.

[8] Yu Du, Fangyun Wei, and Hongyang Zhang. 2024. AnyTool: self-reflective, hierarchical agents for large-scale API calls. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 470, 18 pages.

[9] Aaron Grattafiori, Abhimanyu Dubey, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). https://arxiv.org/abs/2407.21783

[10] Osman Erman Gungor, Derak Paulsen, and William Kang. 2025. Schemora: schema matching via multi-stage recommendation and metadata enrichment using off-the-shelf llms. *arXiv preprint arXiv:2507.14376* (2025).

[11] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, Gaël Varoquaux, Travis Vaught, and Jarrod Millman (Eds.). Pasadena, CA, USA, 11–15.

[12] Benjamin Hättasch, Michael Truong-Ngoc, Andreas Schmidt, and Carsten Binnig. 2022. It's AI Match: A Two-Step Approach for Schema Matching Using Embeddings. arXiv:2203.04366 [cs.DB] https://arxiv.org/abs/2203.04366

[13] IBM Research. 2025. Granite 4.0 Language Models. https://github.com/ibm-granite/granite-4.0-language-models. Apache 2.0 License.

[14] Alistair E. W. Johnson, Tom J. Pollard, Lu Shen, Li-wei H. Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G. Mark. 2016. MIMIC-III, a freely accessible critical care database. *Scientific Data* 3 (2016), 160035. https://doi.org/10.1038/sdata.2016.35

[15] Manish Kesarwani, Sambit Ghosh, Nitin Gupta, Shramona Chakraborty, Renuka Sindhgatta, Sameep Mehta, Carlos Eberhardt, and Dan Debrunner. 2024. GraphQL Query Generation: A Large Training and Benchmarking Dataset. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina (Eds.). Association for Computational Linguistics, Miami, Florida, US, 1595–1607. https://doi.org/10.18653/v1/2024.emnlp-industry.117

[16] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 39–48. https://doi.org/10.1145/3397271.3401075

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2005.11401

[18] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3102–3116.

[19] Zhen Lin, Shubhendu Trivedi, and Jimeng Sun. 2024. Generating with Confidence: Uncertainty Quantification for Black-Box Large Language Models. *Transactions on Machine Learning Research* (2024).

[20] Mingwei Liu, Yanjun Yang, Yiling Lou, Xin Peng, Zhong Zhou, Xueying Du, and Tianyong Yang. 2023. Recommending Analogical APIs via Knowledge Graph Embedding. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1496–1508. https://doi.org/10.1145/3611643.3616305

[21] Yurong Liu, Eduardo Pena, Aecio Santos, Eden Wu, and Juliana Freire. 2024. Magneto: Combining Small and Large Language Models for Schema Matching. arXiv:2412.08194 [cs.DB] https://arxiv.org/abs/2412.08194

[22] Yangyang Lu, Ge Li, Zelong Zhao, Linfeng Wen, and Zhi Jin. 2017. Learning to infer API mappings from API documents. In *Knowledge Science, Engineering and Management: 10th International Conference, KSEM 2017, Melbourne, VIC, Australia, August 19-20, 2017, Proceedings 10*. Springer, 237–248.

[23] Elias Lumer, Pradeep Honaganahalli Basavaraju, Myles Mason, James A. Burke, and Vamse Kumar Subbiah. 2025. Graph RAG-Tool Fusion. arXiv:2502.07223 [cs.CL] https://arxiv.org/abs/2502.07223

[24] Jayant Madhavan, Philip Bernstein, and Erhard Rahm. 2001. Generic Schema Matching with Cupid. *Proc 27th VLDB Conference* (07 2001).

[25] Potsawee Manakul, Adian Liusie, and Mark J.F. Gales. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 9004–9017.

[26] Lakshmi Mandal, Balaji Ganesan, Avirup Saha, and Renuka Sindhgatta. 2024. Graph-Guided API Sequencing with Reinforcement Learning. https://openreview.net/forum?id=svtPejQ35g

[27] Lacramioara Mazilu, Norman W. Paton, Alvaro A.A. Fernandes, and Martin Koehler. 2022. Schema mapping generation in the wild.

[28]]mediawiki MediaWiki Contributors. [n. d.]. MediaWiki. https://www.mediawiki.org. Free software under GPL v2+.

[29] OpenAI, Josh Achiam, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023). https://arxiv.org/abs/2303.08774

[30] J. Marc Overhage, Patrick B. Ryan, Christian G. Reich, Abraham G. Hartzema, and Paul E. Stang. 2012. Validation of a common data model for active safety surveillance research. *Journal of the American Medical Informatics Association* 19, 1 (2012), 54–60. https://doi.org/10.1136/amiajnl-2011-000376

[31] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334* (2023).

[32] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789* (2023).

[33] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics. https://arxiv.org/abs/1908.10084

[34] Avirup Saha, Lakshmi Mandal, Balaji Ganesan, Sambit Ghosh, Renuka Sindhgatta, Carlos Eberhardt, Dan Debrunner, and Sameep Mehta. 2024. Sequential API Function Calling Using GraphQL Schema. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 19452–19458. https://doi.org/10.18653/v1/2024.emnlp-main.1083

[35] Nabeel Seedat and Mihaela van der Schaar. 2025. Bootstrapping Self-Improvement of Language Model Programs for Zero-Shot Schema Matching. In *Forty-second International Conference on Machine Learning*.

[36] Yanjie Shao, Tianyue Luo, Xiang Ling, Limin Wang, and Senwen Zheng. 2022. Cross Platform API Mappings based on API Documentation Graphs. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 926–935.

[37] Eitam Sheetrit, Menachem Brief, Moshik Mishaeli, and Oren Elisha. 2024. ReMatch: Retrieval Enhanced Schema Matching with LLMs. arXiv:2403.01567 [cs.DB] https://arxiv.org/abs/2403.01567

[38] Jason Walonoski, Mark Kramer, Joseph Nichols, Andre Quina, Chris Moesel, Dylan Hall, Carlton Duffett, Kudakwashe Dube, Thomas Gallagher, and Scott McLachlan. 2018. Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association* 25, 3 (2018), 230–238. https://doi.org/10.1093/jamia/ocx079

[39] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2614–2627. https://doi.org/10.1145/3448016.3457550

[40] Sha Wang, Yuchen Li, Hanhua Xiao, Bing Tian Dai, Roy Ka-Wei Lee, Yanfei Dong, and Lambert Deng. 2025. LLMATCH: A Unified Schema Matching Framework with Large Language Models. In *Asia-Pacific Web (APWeb) and Web-Age*

Information Management (WAIM) Joint International Conference on Web and Big Data. Springer, 343–356.

[41] Quan Xiao, Debarun Bhattacharjya, Balaji Ganesan, Radu Marinescu, Katsiaryna Mirylenka, Nhan H. Pham, Michael Glass, and Junkyu Lee. 2025. The Consistency Hypothesis in Uncertainty Quantification for Large Language Models. In *Proceedings of the Forty-First Conference on Uncertainty in Artificial Intelligence (UAI '25)*. JMLR.org. arXiv:2506.21849.

[42] Yunjia Zhang, Avrilia Floratou, Joyce Cahoon, Subru Krishnan, Andreas C. Müller, Dalitso Banda, Fotis Psallidas, and Jignesh M. Patel. 2023. Schema Matching using Pre-Trained Language Models. , 1558-1571 pages. https://doi.org/10.1109/ICDE55515.2023.00123

[43] Yu Zhang, Di Mei, Haozheng Luo, Chenwei Xu, and Richard Tzong-Han Tsai. 2025. SMUTF: Schema Matching Using Generative Tags and Hybrid Features. *Information Systems* 133 (2025), 102570. https://doi.org/10.1016/j.is.2025.102570

[44] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2306.05685